

Projektbeskrivning

<Chess XD>

Projektmedlemmar:

Elliot Norlander <ellno907@student.liu.se>

Elias Bergsman <elibe559@student.liu.se>

Handledare:

Daniel Pettersson <danpe975@student.liu.se>

Innehåll

1. Introduktion till projektet	2
2. Ytterligare bakgrundsinformation.....	2
3. Milstolpar	2
4. Övriga implementationsförberedelser	3
5. Utveckling och samarbete.....	3
6. Implementationsbeskrivning.....	4
6.1. Milstolpar	4
6.2. Dokumentation för programstruktur, med UML-diagram	4
7. Användarmanual	8

Projektplan

1. Introduktion till projektet

Målet med detta projekt är att skapa en fungerande version av det klassiska brädspelet schack för att sedan generera en grafisk representation av spelet.

2. Ytterligare bakgrundsinformation

Målet med spelet är att försätta motståndarens kung i schackmatt, det vill säga att motståndarens kung inte kan röra sig utan att bli tagen av en av motståndarens pjäser. I schack finns det sex olika typer av pjäser som kan röra sig på olika sätt. Pjäserna är kung, drottning, springare, torn, löpare och bonde. Reglerna i schack är välkända, och finns att tillgå på till exempel <https://www.chess.com/sv/schackregler>.

3. Milstolpar

#	Beskrivning
1	Skapa en Square – Dvs en ruta på brädet
2	Skapa ett bräde bestående av 8 x 8 rutor, samt visa detta på skärmen
3	Skapa pjäsen "bonde" så den följer spelreglerna
4	Skapa pjäsen "löpare" så den följer spelreglerna
5	Skapa pjäsen "springare" så den följer spelreglerna
6	Skapa pjäsen "torn" så den följer spelreglerna
7	Skapa pjäsen "drottning" så den följer spelreglerna
8	Skapa pjäsen "kung" så den följer spelreglerna
9	Implementera en funktion som undersöker hur en pjäs får flytta sig givet ett visst spelläge
10	Implementera en funktion som flyttar runt pjäser på brädet
11	Implementera spelarkontroller så att en spelare kan flytta en pjäs på brädet
12	Implementera spelarkontroller för en andra spelare
13	Implementera andra spelmekanismer:

14	i)	Timer
15	ii)	Vinstvillkor
16	iii)	En omstartsknapp
17	Implementera bytet mellan bonde och drottning	
18	Implementera rockad och dess villkor	
19	*UI/Grafiska element kommer utvecklas kontinuerligt under projektets gång*	

4. Övriga implementationsförberedelser

Kärnan bakom implementationen av detta spel är klassindelning och objektorienterad programmering. Exempel på detta är samverkan mellan en enskild ruta på brädet, brädet i stort och vilken typ av pjäs som står på rutan.

Detta är själva grunden i brädets konstruktion. Planen är också att dela in varje typ av pjäs i en egen klass, och på så sätt skapa den variation av pjäser som krävs för ett fungerande schackspel.

5. Utveckling och samarbete

Vad gällande ambitionsnivå, arbetsinsats och arbetssätt är projektets parter överens.

Målet är att få detta projekt godkänt till första inlämningstillfället, dock anses det vara tillåtet att ta mer tid än så för att lämna in ett slutgiltigt projekt som båda parter är stolta över.

Nedan följer en översiktlig planering av projektet.

Veckonummer	Fokusområde
13	Den enskilda rutan och spelbrädet
14	Konstruktion av pjäser
15	Konstruktion av pjäser och implementation av förflyttning
16	Implementation av förflyttning
17	Spelkontroller såsom timers, schackmatt och rockad
18	Fortsättning av spelkontroller
19	Avbuggning och förfining
20	Redovisning

Vad gäller fördelningen av kodskrivande tänker vi att vi varvar gemensamt och enskilt arbete. Vissa saker, såsom spelbrädet, anser vi är så pass centrala att det är bra om båda parter är med och utvecklar dem.

Projektrapport

6. Implementationsbeskrivning

I följande avsnitt beskrivs spelets struktur och funktionalitet.

6.1. Milstolpar

Under planeringsfasen av projektet skapades nitton milstolpar. Poängen med dessa var att underlätta implementationen av programmet.

Vad gällande milstolpe ett till tolv har dessa implementerats till fullo. Milstolparna och dess relaterade funktioner är centrala i schackspelet och är därav nödvändiga att implementera. Vidare bör det nämnas att milstolpe tretton till arton inte är fullständigt implementerade ännu. Anledningen är att spelets grafiska element visat sig svårare än väntat att utveckla.

6.2. Dokumentation för programstruktur, med UML-diagram

Programmets kärna finns i klassen "Game". Det är denna fil som kontrollerar huruvida spelet körs eller inte. Detta sker med hjälp av en huvudmetod i filen. En metod i programmeringsspråket Java, är en funktion som utför en eller flera rader kod när metoden kallas på. I detta fall används en huvudmetod, eller en så kallad main-metod. Det är existensen av denna huvudmetod som dikterar huruvida programmet är körbart eller inte.

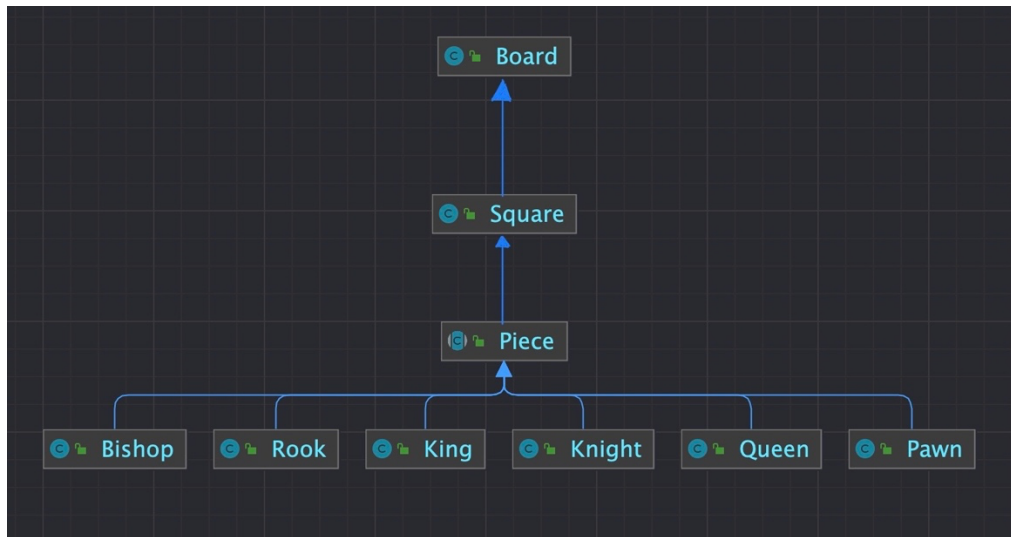
När Gamefilen körs kommer en rad saker att ske. Det första som sker är en tillkallning till filen StartMenu. Här skapas en startmeny och visas på skärmen. Detta görs med hjälp av funktioner från det externa klassbiblioteket Swing. Ett externt klassbibliotek hjälper programmeraren genom att tillhandahålla nya funktioner som är redo att implementeras i koden. Det kan röra sig om till exempel alternativa sätt att behandla olika datatyper. I det berörda programmets fall tillhandahåller Swing en rad olika gränssnittsverktyg som möjliggör bildgenerering och konstruktion av menyn i sig.

När spelaren trycker på startknappen på startmenyn startas spelet. Det första som sker är att 64 objekt av klassen Square skapas. Här tilldelas objekten, det vill säga rutorna, en x- och y-koordinat och en färg. Detta görs i board-klassens huvudkonstruktor. En konstruktor är en typ av metod som tilldelar värden eller tillstånd till olika datafält¹ i en klass. Denna tilldelning är nödvändigt vid skapandet av rutorna som bildar schackbrädet, då rutor med samma koordinater inte efterfrågas.

Rutorna tilldelas en pjäs med hjälp av metoden initPieces. Vilken ruta som får vilken pjäs dikteras av pjäsernas respektive startplats. Här används först en iterativ lösningsmetod för att placera ut vita och svarta bönder. Därefter placeras resterande pjäser ut på brädet. Detta sker genom tilldelning av tillstånd i varje pjäsobjekts datafält.

¹ https://www.mathworks.com/help/matlab/matlab_external/data-fields-of-java-objects.html

Metoden `initPieces` tillkallar metoden `setPiece` som återfinns i klassen `Square`. Denna metod tilldelar en specifik ruta med en pjäs, detta sker genom att uppdatera datafältet, `ocPiece`, med en ockuperande pjäs. Likt sättet i figur 1 skapas en slags växelverkan mellan brädet, rutan och den enskilda pjäsen



Figur 1, illustrerande UML-diagram över samspel mellan `Board`, `Square`, `Piece` samt subklasser till `Piece`

I början av klassen `Board` finns två så kallade for-loopar. En for-loop är en algoritm som används för att iterera genom ett eller flera steg av kod. Detta är effektivt då en vill utföra samma eller snarlika operationer på liknande typer av data. I detta fall vill vi iterera genom en lista av index åtta där vi lägger till en ruta för varje index. Här uppstår ett problem; varannan ruta ska vara svart respektive vit. Detta problem löses genom en så kallad if-sats. En if-sats kan liknas vid svenskans "om", då den helt enkelt avgör huruvida ett eller flera villkor är uppfyllda. Om villkoren är uppfyllda, kommer koden under satsen köras, annars går datorn vidare utan att köra den underställda koden. I klassen `Boards` fall används detta genom att läsa av varje enskild rutas färgfält, för att sedan avgöra huruvida rutan ska vara svart eller vit.

Vad gällande de grafiska elementen av brädet hanteras dessa bland annat av metoden `paintComponent` i klassen `Square`. I denna metod avläses först datafältet `colour` för varje enskilt `Square`objekt. Om fältet innehåller en etta, färgas den utritade rutan med en gulvit färg. Annars fylls rutan med en mörkgrön färg.

För att rita ut spelpjäserna på brädet används `Piece`-klassens huvudkonstruktor. Här tilldelas datafältet `imageIcon` en URL-länk till bilden av sig själv. Denna konverteras sedan till en bild av typen `BufferedImage`. Själva utskriften av pjäsen sker i metoden `drawIt`. Metoden läser av vart den ska skriva ut pjäsen med hjälp av pjäsens koordinater. Därefter används den inbyggda metoden `drawImage` för att skriva ut pjäsen på skärmen.

Varje typ av pjäs har sin egen klass. Alltså finns det individuella klasser för kung, dam, torn, löpare, springare och bonde. Här tillämpas en form av polymorfism, då alla dessa

klasser härstammar från klassen Piece. Här fungerar klassen Piece som en slags mall, och varje objekt av typen kung, dam, etcetera får många av sina egenskaper från just Piece-klassen. Denna klass är en så kallad abstrakt klass. Det innebär att klassen som sådan inte kan instansieras. Däremot kan den utökas med hjälp av subklasser. På detta sätt sparas programminne, vilket bidrar till ett snabbare och mer lättkört program. I fallet Piece är abstraktion mycket fördelaktig då pjäserna har flera likheter gällande funktionalitet och attribut.

I klassen Piece regleras stora delar av pjäsernas förflyttning. Detta utförs främst i metoden `move`. Denna metod undersöker först huruvida rutan, som den förflyttade pjäsen försöker förflytta sig till, är tom eller inte. Om rutan är tom, och den förflyttande pjäsen får röra sig till den tomma rutan, förflyttas pjäsen dit genom att datafältet `currentPosition` uppdateras med en ny koordinat. I de fallen där rutan redan är ockuperad undersöker programmet först huruvida rutan ockuperas av en motståndarens pjäser. Om så är fallet kallar förflyttningsmetoden på metoden `captureToKill` som då eliminerar motståndarpjäsen helt och hållet, därefter uppdateras pjäsens koordinater.

I metoden `captureToKill` hanteras som tidigare nämnt övertagandet av motståndarpjäser. Här används `if`-satser för att avgöra huruvida övertagandet är möjligt eller inte. Om `if`-satsernas villkor är uppfyllda uppdateras rutan, i vilken den nu döda pjäsen stod, med en ny pjäs. Den döda pjäsen tas därefter bort från listan av spelarpjäser.

I denna version av schack finns de sex olika pjäser som spelet är känt för. De skapas alla med hjälp av den abstrakta klassen Piece och placeras med hjälp av klassen Board. Varje enskild pjäs har förstås ett antal egna egenskaper som skiljer sig mellan olika pjäser. Pawn-klassen är speciell då dess möjliga drag dels beror på om pjäsen har flyttats från sin startposition. Vidare spelar det roll om det står en motståndarpjäs i närheten. Om denna motståndarpjäs står framför, bakom eller diagonalt i förhållande till bonden påverkar även pjäsens möjligheter till förflyttning. Av denna anledning implementerades ett datafält, `movedPiece`, som indikerar huruvida en bonde har förflyttats eller inte. Om det booleska uttrycket är sant, har pjäsen tidigare förflyttats. Denna booleska variabel kan sedan användas i kombination med `if`-satser för att bestämma hur en specifik bonde kan röra sig.

Som tidigare nämnt är Piece-klassen ett typexempel på abstraktion inom java. Ytterligare ett steg i denna process är användningen av `overrides`. I Piece-klassen återfinns metoden `fetchLegalMoves`. Denna metod undersöker alla tillåtna drag som varje enskild pjäs kan göra i en viss tidpunkt. Implementationen i Piece-klassen är dock blank, det vill säga att den inte innehåller någon kod. Anledningen är att det i varje subklass av Piece finns en överskridande metod med samma namn. Här utnyttjas alltså `overrides` för att skapa abstraktion i form av en gemensam funktion. Denna funktion utför olika saker beroende på vilken klass som är aktuell för tillfället.

Att de olika pjäserna rör sig på olika sätt är den huvudsakliga anledningen till att det lämpar sig att använda denna form av abstraktion. Om alla pjäser hade förflyttat sig identiskt hade en gemensam metod varit mer fördelaktig. Men eftersom så inte är fallet behövs det, beroende på vilken pjäs det handlar om, olika aritmetiska och logiska

uttryck för att bestämma hur pjäsen kan röra sig.

Som exempel på ovanstående abstraktion har vi metoden `fetchLegalMoves` i klassen `Pawn`. Med hjälp av `if`-satser kan de lagliga rörelserna undersökas för både de vita och svarta bönderna. Dessa, till skillnad från de andra pjäserna, skiljer sig beroende på vilken färg de har. Anledningen till detta är att pjäsen bonde endast får röra sig framåt med avseende på brädets `y`-axel. Här används aritmetik och logik för att bestämma bondens tillåtna rörelser. De tillåtna dragen läggs därefter till i listan `legalMoves`, som sedan används för att undersöka huruvida en pjäs får flytta sig till en efterfrågad ruta.

I `Board`-klassen stöter vi redan i första raden på ett begrepp som har att göra med en av grundpelarna bakom språket `java` och programmeringsparadigmet objektorientering. `“Public class Board extends JPanel”` medför att vår klass ärver egenskaperna hos `JPanel`. Med andra ord kan vi nu ta del av den funktionalitet vi vill komma åt i `JPanel`, men också bygga ut och anpassa önskvärda delar i `Board`. Detta implementeras i form av kod i `Board`-klassen. Nyckelordet är i detta fall `“extends”`, då denna fras berättar för datorn att `Board` ska ärva de egenskaper som `JPanel` har.

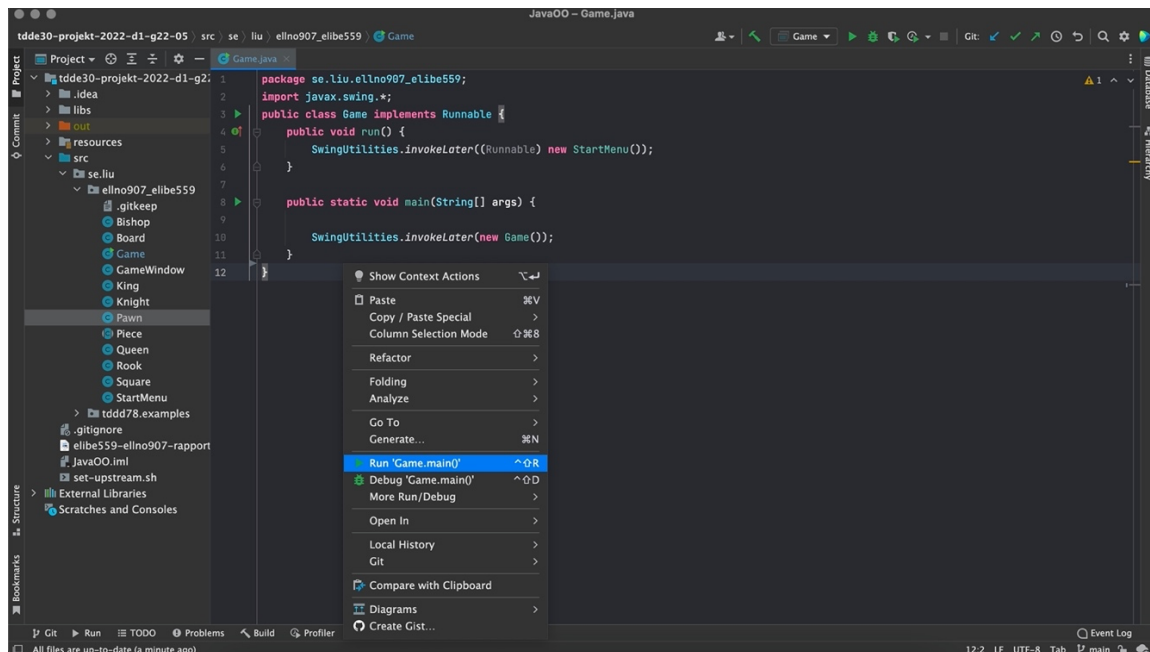
I denna implementation av schack representeras brädet på två sätt. Först och främst finns det logiska sättet. Det är här som spelets funktionalitet beskrivs och hanteras. Det är denna representation av brädet som är kärnan i spelet. Vidare finns det också en grafisk representation i form av ett grafiskt användargränssnitt, även kallat GUI². Det är detta grafiska gränssnitt som användaren ser på sin skärm när programmet körs.

Den logiska delen av brädet representeras i form av datafältet `board`. `Board` är en tvådimensionell array som markerats som `private final`, vilket innebär att fältet inte kan ändras på när det väl har tilldelats ett värde. Här finns ytterligare ett exempel på en central term inom objektorienterad programmering, nämligen inkapsling. Inkapsling innebär att man undanhåller data från andra klasser. Det medför minskad risk för felmanipulering samt att en klass kan ha full kontroll över vilken data som lagras i dess fält. För att komma åt den undanhållna informationen får utomstående klasser anropa klassens getters och setters. Dessa metoder kan i sin tur hämta och tilldela information i datafälten.

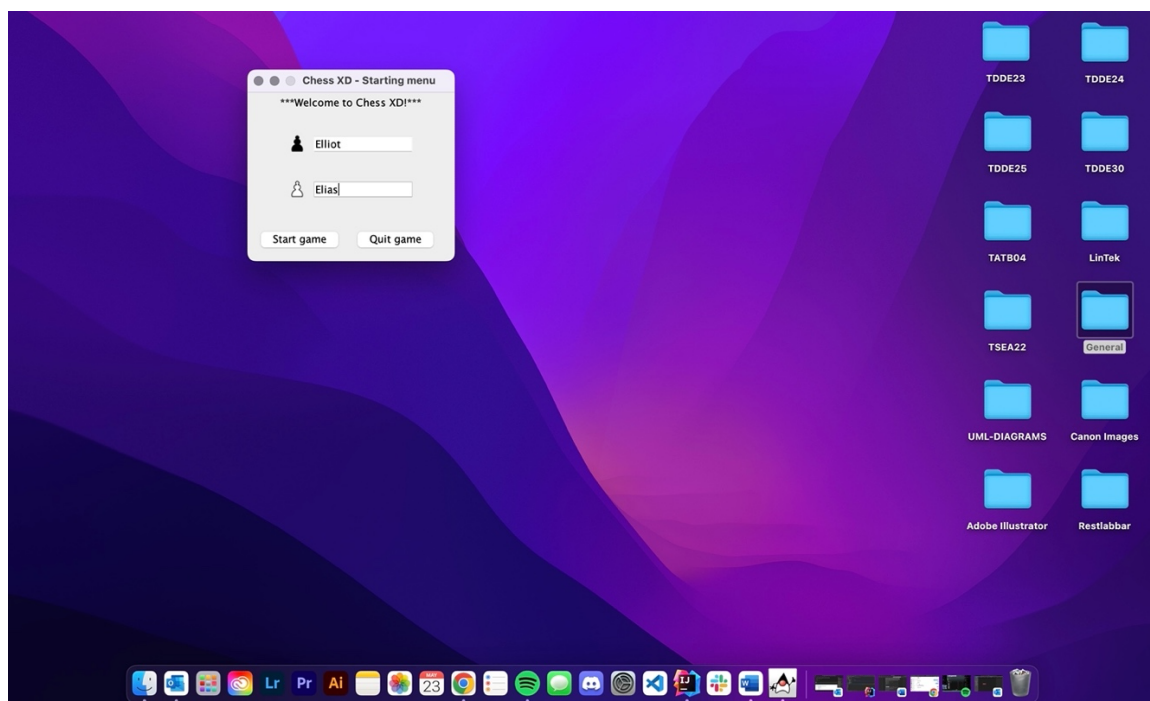
² GUI – Graphical User Interface

7. Användarmanual

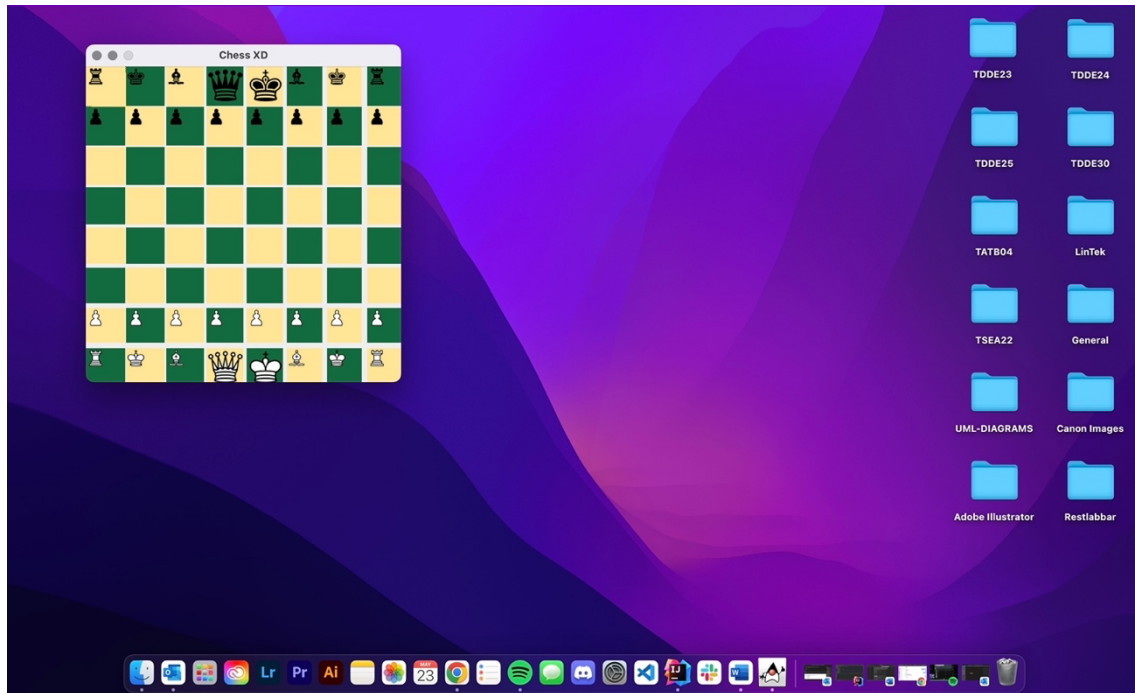
Innan du startar spelet måste användaren ha tillgång till hela paketet med alla nödvändiga filer, klassbibliotek och bilder. Detta innefattar mapparna SRC och Resources. Därefter, öppna projektet i valfri IDEA och kör filen Game.java.



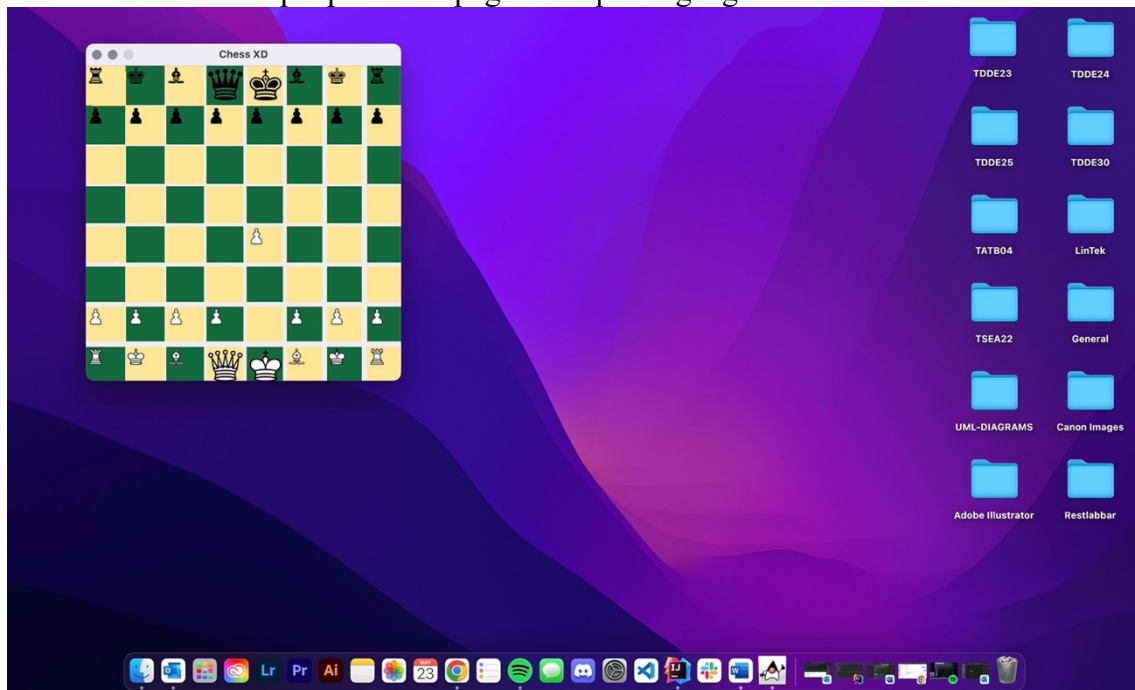
Nu kommer en startmeny visas på skärmen. Fyll i spelarnas namn genom att klicka och sen skriva i de två textfälten. Tryck därefter på knappen "Start game" för att starta spelet, eller tryck "Quit game" för att avsluta.



Nu har spelet startats. För att flytta pjäserna, klicka på en pjäs, håll nere, och dra pjäsen till önskad ruta. Om draget inte är tillåtet kommer pjäsen flyttas tillbaka till sin ursprungsposition.



Nedan finns ett exempel på hur en pågående spelomgång kan se ut.



Spelet följer de standardiserade reglerna i schack. De finns att läsa på <https://www.chess.com/sv/schackregler>.